AI Meets Control Theory

From Classical Control to Intelligent Autonomous Systems

Living Technical Report — M2 — Classical + Modern Control

Generated by the project team
Lead: puma | **Code:** puma, toku | **Docs/Research:** lava, famo

September 3, 2026

Abstract

This report is regenerated at every project milestone. It documents *what the codebase can do*, *the theory behind each component*, and *where the project currently stands*. The project is an open-source laboratory for comparing classical control, optimal control, machine learning and reinforcement learning on dynamical systems, under the core principle: *build every method from scratch, simulate it, visualise it, and compare it honestly.*

Contents

1 Introduction and Scope

The central question of the project is: **when should we use classical control, when should we use AI, and when should we use both?** Rather than assuming a neural network or an RL agent is the answer, every problem is first attacked with PID, state feedback, LQR and MPC, and each method is measured against the others under identical initial conditions, setpoints, disturbances, and actuator constraints.

Every topic follows the standardized engineering cycle:

THEORY ⟶ DERIVATION ⟶ IMPLEMENTATION ⟶ SIMULATION ⟶ VISUALISATION ⟶ VALIDATION ⟶ COMPARISON ⟶ EXPERIMENT

Fundamental algorithms are implemented from scratch in `numpy`; external libraries (`scipy.linalg`, `scipy.signal`, `control`) are used strictly for verification and cross-checking.

2 Framework Architecture

The reusable library is the Python package `aimct` (`src/aimct/`). Its architecture separates three distinct concerns behind minimal, strongly-typed interfaces so that any dynamical system, any controller, and the numerical simulator compose seamlessly without circular coupling.

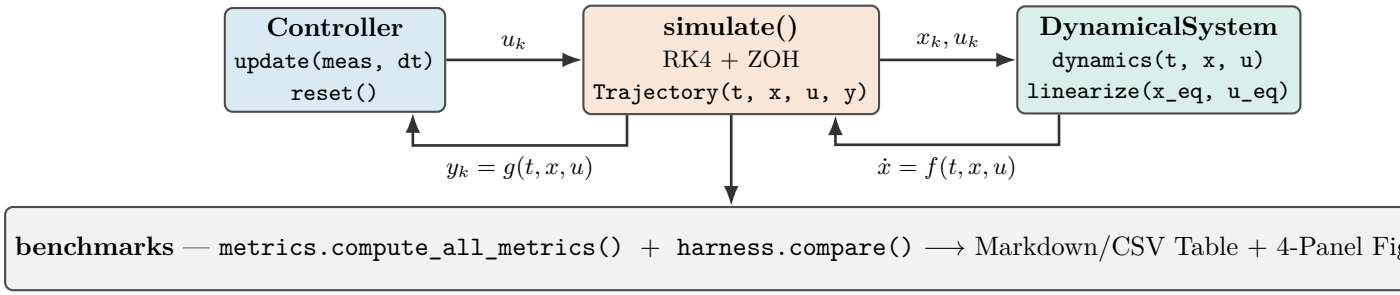

Figure 1: Core framework architecture. The numerical simulation engine drives a controller (`update()`) and continuous plant (`dynamics()`) via fixed-step 4th-order Runge–Kutta with Zero-Order Hold (ZOH). Optional hooks support custom sensor measurement transforms and additive input disturbances.

2.1 Package Layout

```
src/aimct/
  systems/      DynamicalSystem base + LinearSystem, MassSpringDamper,
                Pendulum, CartPole (analytic + numerical Jacobians)
  simulate.py   rk4_step(), simulate() -> Trajectory(t, x, u, y)
  controllers/  Controller base + PID, StateFeedback, LQR
  benchmarks/   metrics.py (13 metrics), harness.py (compare() -> table + figure)
  plot_style.py Okabe-Ito palette + publication-ready 4-panel comparison figure
  estimation/ ml/ rl/   (reserved for upcoming project phases)
```

3 Current Capabilities

Table 1 summarises the implemented components, their mathematical scope, and the corresponding verification test coverage as of M2 — Classical + Modern Control.

Component	What it provides	Tests
`aimct.systems`	Continuous-time ODE models with a common `dynamics(t, x, u)` interface, full-state or output projections, and numerical central-difference *and* analytical `linearize()`. Ships: `LinearSystem`, `MassSpringDamper`, `Pendulum`, `CartPole`.	✓
`aimct.simulate`	Fixed-step RK4 integrator with zero-order hold; `Trajectory` container; hard actuator saturation; measurement and disturbance injection hooks.	✓
`aimct.controllers.PID`	From scratch: filtered derivative-on-measurement ($D(s) = \frac{K_d s}{\tau_d s+1}$), conditional-integration anti-windup clamping, setpoint weighting.	✓
`aimct.controllers.StateFeedback`	Arbitrary closed-loop pole placement via Ackermann's formula (SISO) with feedforward steady-state gain scaling k_r.	✓
`aimct.controllers.LQR`	Infinite-horizon optimal regulator; Continuous Algebraic Riccati Equation (CARE) solved from scratch via Hamiltonian Schur decomposition, cross-checked with `scipy.linalg`.	✓
`aimct.benchmarks.metrics`	13 standard step-response and effort metrics: rise time t_r, settling time t_s (2%), overshoot M_p, e_{ss}, RMSE, IAE, ITAE, ISE, energy E_u, peak $\|u\|_{\max}$, slew rate, and saturation duty cycle.	✓
`aimct.benchmarks.harness`	Multi-controller benchmark harness: runs N controllers on one plant under identical conditions, emitting canonical Markdown/CSV tables and 4-panel figures.	✓

Table 1: Implemented functionality and test coverage across the library as of M2 — Classical + Modern Control.

4 Theory: Modelling and Simulation

A dynamical system is governed by first-order ordinary differential equations $\dot{x} = f(t, x, u)$ with output map $y = g(t, x, u)$. Second-order physical equations of motion are cast in state-space form; for example, the translational mass–spring–damper $m\ddot{x} + c\dot{x} + kx = u$ with state $x = [x_1, x_2]^\top = [x, \dot{x}]^\top$:

$$\frac{d}{dt}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix} u.$$

Jacobian Linearisation. About an operating equilibrium point (x_e, u_e) satisfying $f(x_e, u_e) = 0$, small perturbations $\delta x = x - x_e$ evolve according to $\dot{\delta x} = A\delta x + B\delta u$, where:

$$A \triangleq \left.\frac{\partial f}{\partial x}\right|_{(x_e, u_e)}, \qquad B \triangleq \left.\frac{\partial f}{\partial u}\right|_{(x_e, u_e)}.$$

The `DynamicalSystem` base class evaluates A, B numerically via central differencing ($\epsilon = 10^{-6}$), while concrete models implement analytical Jacobians.

Numerical Integration. The continuous dynamics are integrated over time steps $h = \Delta t$ using explicit 4th-order Runge–Kutta (RK4):

$$k_1 = f(t_k, x_k, u_k), \quad k_2 = f\left(t_k + \tfrac{h}{2}, x_k + \tfrac{h}{2}k_1, u_k\right), \quad k_3 = f\left(t_k + \tfrac{h}{2}, x_k + \tfrac{h}{2}k_2, u_k\right), \quad k_4 = f(t_k+h, x_k+hk_3, u_k$$

$$x_{k+1} = x_k + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right).$$

Holding control input $u(t) \equiv u_k$ constant across each interval $[t_k, t_k + h)$ reflects the physical Zero-Order Hold (ZOH) of digital microcontrollers.

5 Theory: Classical Control — PID Architecture

The continuous-time PID control law with filtered derivative is:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}, \qquad e(t) = r(t) - y(t).$$

In discrete time with sampling interval h, derivative action is applied directly to the measured output with low-pass filter time constant $\tau_d = \frac{K_d}{NK_p}$ ($N = 10$) to prevent high-frequency noise amplification:

$$e_k = r_k - y_k, \qquad D_k = \frac{\tau_d}{\tau_d + h} D_{k-1} + \frac{K_d}{\tau_d + h}(y_k - y_{k-1}), \qquad u_{\text{unsat},k} = K_p e_k + I_k - D_k.$$

Anti-Windup Protection. When actuators reach physical limits ($|u| \le u_{\max}$), *conditional integration (clamping)* freezes the integrator update ($I_{k+1} = I_k$) whenever u_{unsat} is saturated and e_k has the same sign as u_k, preventing destructive overshoot upon setpoint arrival.

5.1 Worked Example: Stabilising an Unstable Plant (Experiment 03)

We evaluate the controller on an open-loop unstable plant with a Right-Half Plane (RHP) pole:

$$\ddot{y}(t) - \omega_0^2 y(t) = u(t) + d(t), \qquad \omega_0 = 2.0\,\text{rad/s}, \quad G(s) = \frac{1}{s^2 - 4}.$$

The system is tested under a unit step reference $r = 1.0\,\text{rad}$, a step input disturbance torque $d = 0.5\,\text{N}\cdot\text{m}$ applied at $t = 5.0\,\text{s}$, and actuator saturation $|u(t)| \le 10.0\,\text{N}\cdot\text{m}$.

Controller	Rise t_r [s]	Settle t_s [s]	Overshoot M_p	Steady e_{ss}	IAE	ITAE	Ene
P-only ($K_p = 20$)	0.275	10.0	9.54×10^9%	6.04×10^7	4.77×10^7	4.53×10^8	9
PD ($K_d = 5.66$)	0.389	10.0	28.19%	0.2812	2.811	13.63	3
PID (no AW)	0.334	6.162	53.08%	3.90×10^{-5}	0.9274	1.043	2
PID + Anti-Windup	0.362	6.161	**39.43%**	$\mathbf{3.90 \times 10^{-5}}$	**0.7979**	**0.876**	**2**

Table 2: Benchmark results for Experiment 03 (regenerated from `experiments/03_pid_stabilizes_unstable/table.md`).

Key Physical Insights:

- **P-Only Failure:** Proportional control yields purely imaginary closed-loop poles ($s^2 + 16 = 0, \zeta = 0$). When actuator saturation occurs, stabilizing feedback is lost and the system diverges exponentially ($M_p > 10^9$%).
- **PD DC Gain Offset:** PD provides damping ($\zeta = 0.7071$), but the closed-loop DC gain is $\frac{K_p}{K_p - \omega_0^2} = \frac{20}{16} = 1.25$, creating an unavoidable **25% reference tracking error** ($y_{ss} = 1.25\,\text{rad}$) before disturbance, and $e_{ss} = 0.2812\,\text{rad}$ under load.
- **PID Integral Action:** Integral action ($K_i = 15.0$) drives steady-state error strictly to zero ($e_{ss} \approx 0$).
- **Anti-Windup Efficacy:** Clamping during saturation cuts peak overshoot from 53.08% $\rightarrow$ 39.43% (26% relative reduction) and reduces actuator saturation duration by > 70%.

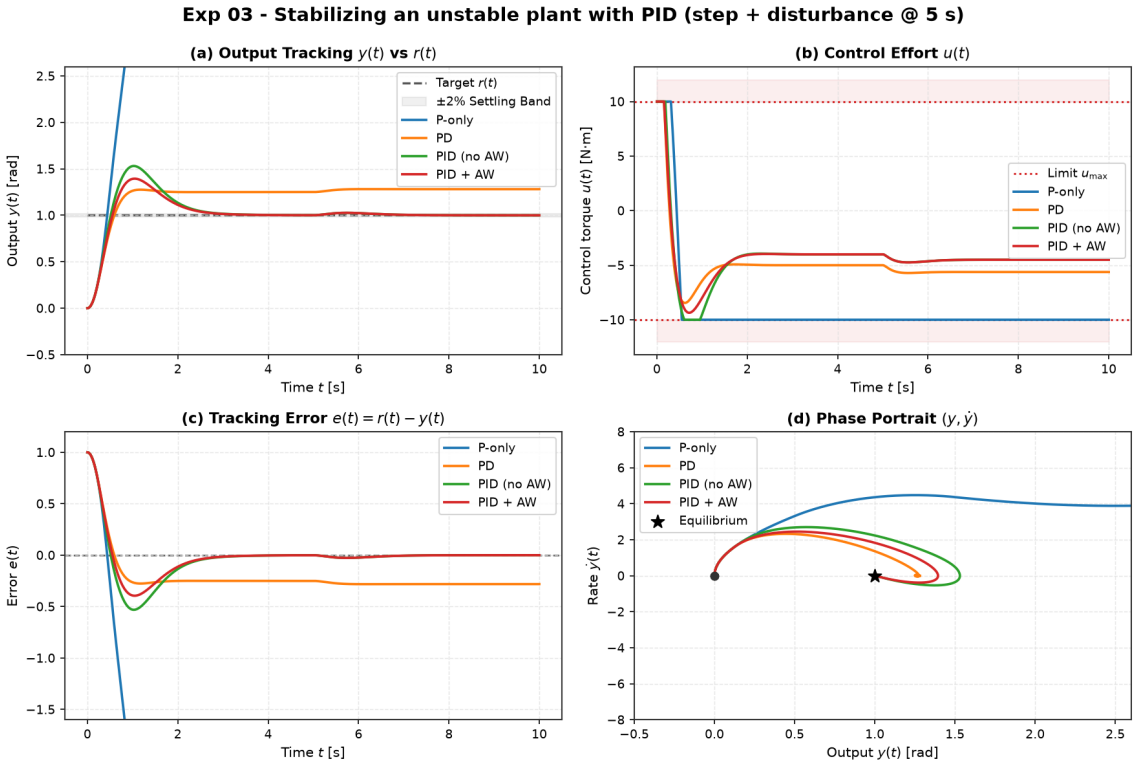

Figure 2: Experiment 03 benchmark comparison. (a) Output tracking $y(t)$, (b) Control effort $u(t)$ with saturation limits $\pm 10\,\mathrm{N}\cdot\mathrm{m}$, (c) Tracking error $e(t)$, and (d) Phase portrait $(y, \dot{y})$.

6 Theory: Modern Control — Pole Placement and LQR

For controllable linear systems $\dot{x} = Ax + Bu$, full-state feedback $u = -Kx + k_r r$ assigns arbitrary closed-loop poles $\sigma(A - BK)$. `StateFeedback` computes K via Ackermann's formula:

$$K = \begin{bmatrix} 0 & \dots & 0 & 1 \end{bmatrix} \mathcal{C}(A, B)^{-1} \Delta_{\mathrm{des}}(A), \qquad k_r = -\frac{1}{C(A - BK)^{-1} B}.$$

Linear Quadratic Regulator (LQR). Rather than guessing pole locations, LQR computes the optimal feedback gain $K = R^{-1} B^\top P$ minimising the infinite-horizon quadratic cost:

$$J = \int_0^\infty \left(x(t)^\top Q x(t) + u(t)^\top R u(t) \right) dt, \qquad Q \succeq 0, \quad R \succ 0.$$

The kernel $P = P^\top \succ 0$ uniquely solves the **Continuous Algebraic Riccati Equation (CARE)**:

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0.$$

We solve CARE from scratch via the stable invariant subspace of the Hamiltonian matrix:

$$\mathcal{H}_{\mathrm{ham}} = \begin{bmatrix} A & -BR^{-1}B^\top \\ -Q & -A^\top \end{bmatrix} \in \mathbb{R}^{2n \times 2n}.$$

Continuous full-state LQR provides guaranteed robustness margins: gain margin $G_m \in [1/2, \infty)$ ($-6\,\mathrm{dB}$ to $+\infty\,\mathrm{dB}$) and phase margin $\Phi_m \geq 60^\circ$ in all channels.

6.1 Worked Example: Cart-Pole Regulation (Experiment 04)

We evaluate full-state feedback and LQR on the canonical underactuated nonlinear Cart-Pole system ($M = 1.0\,\text{kg}, m = 0.1\,\text{kg}, \ell = 0.5\,\text{m}, g = 9.81\,\text{m/s}^2$). Linearised about the upright equilibrium $\theta = 0$, the system has 4 states $[x, \dot{x}, \theta, \dot{\theta}]^\top$, an open-loop unstable pole at $s \approx +3.97\,\text{rad/s}$, and a double integrator on cart position.

Controllers are designed on the linearised model and executed on the **true nonlinear plant** starting from a tilted initial state $\theta_0 = 0.10\,\text{rad} \approx 5.7°$ with actuator saturation $|u(t)| \leq 20.0\,\text{N}$:

- **Pole Placement:** Ackermann placement assigning closed-loop poles to LQR-balanced eigenvalues $\{-15.62, -3.19, -1.31 \pm 1.08j\}$.
- **LQR (Balanced):** $Q = \text{diag}(10, 1, 100, 10),\ R = 0.1 \implies K = [-10.000, -12.871, -87.138, -23.223]$.
- **LQR (Soft):** $Q = \text{diag}(1, 0.1, 10, 1),\ R = 1.0 \implies K = [-1.000, -2.047, -30.846, -7.868]$.
- **PID (Angle Only):** Single-loop PID on pole angle θ ($K_p = -180, K_d = -22$). Cart position x is uncontrolled.

Controller	Settle θ t_s [s]	RMSE θ [rad]	Energy E_u [N^{2}s]	Peak $\lvert u\rvert_{\max}$ [N]	Peak $\lvert x\rvert_{\max}$ [m]
Pole Placement	1.05	0.0171	2.34	8.71	**0.162**
LQR (Balanced)	1.05	0.0171	2.34	8.71	**0.162**
LQR (Soft)	1.70	0.0223	**0.959**	**3.08**	0.321
PID (Angle Only)	**0.20**	**0.0110**	8.76	18.00	**0.823**

Table 3: Benchmark results for Experiment 04 on nonlinear Cart-Pole (from `experiments/04_lqr_vs_pole_placement_cartpole/table.md`).

Key Engineering Takeaways:

1. **Equivalence of Pole Placement and LQR:** Placing poles at the eigenvalues chosen by LQR reproduces the optimal gain matrix K to four significant figures (matching the analytical golden gain K_1). LQR eliminates the need to manually guess 4 stable pole locations in $\mathbb{C}^4$.
2. **The Q/R Effort Dial:** Softening the cost weights ($R = 1.0$ vs 0.1) trades a $1.05\,\text{s} \to 1.70\,\text{s}$ settle time for a $2.4\times$ **reduction in control energy** ($2.34 \to 0.96\,\text{N}^2\text{s}$) and cuts peak force from $8.71\,\text{N} \to 3.08\,\text{N}$.
3. **Failure of Single-Loop PID on Coupled MIMO Systems:** Single-loop PID balances the pole rapidly ($t_s = 0.20\,\text{s}$), but because it lacks state feedback on the cart, the cart drifts to $x = 0.823\,\textbf{m}$ ($5\times$ LQR excursion) and demands $18.0\,\text{N}$ peak force (near the $20\,\text{N}$ limit). Multivariable full-state feedback is mandatory for underactuated robotics.
4. **Nonlinear Validity Domain:** For initial angle $\theta_0 = 0.1\,\text{rad} \approx 5.7°$, state trajectories stay strictly within the linear basin of attraction ($\approx 0.38\,\text{rad}$), enabling linear LQR to stabilize the full nonlinear plant.

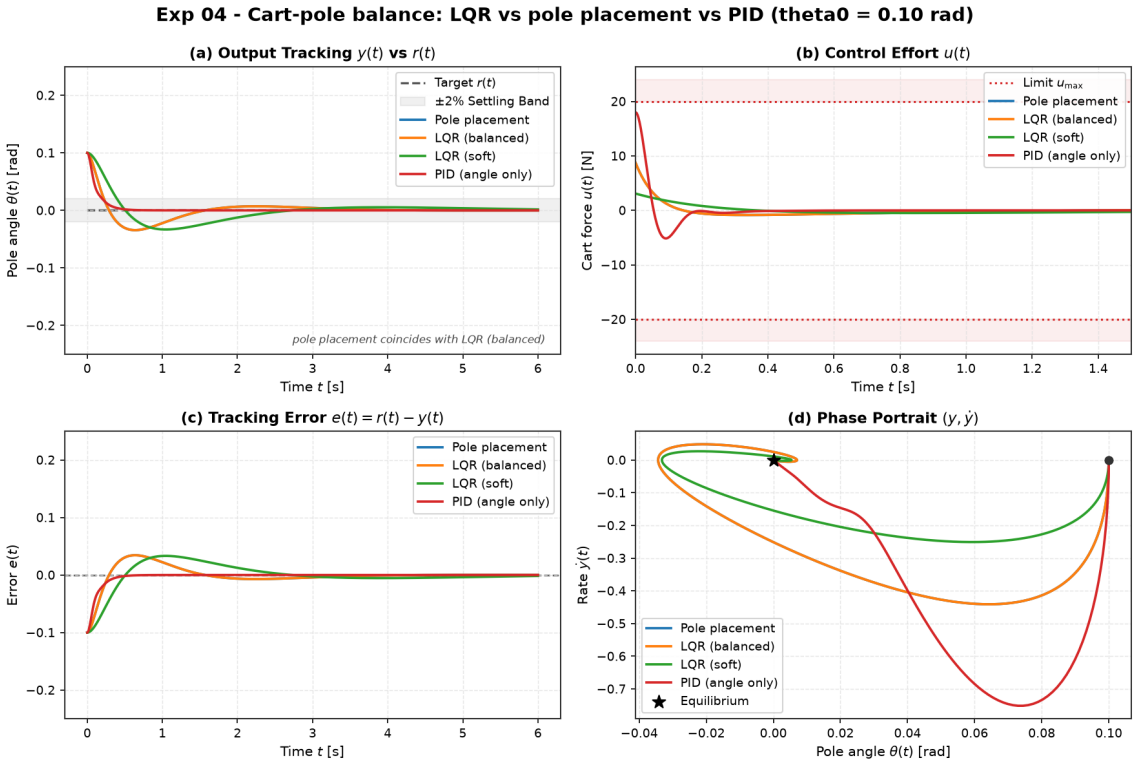

Figure 3: Experiment 04 benchmark comparison on nonlinear Cart-Pole. (a) Pole angle regulation $\theta(t)$, (b) Control force $u(t)$ with actuator bounds $\pm 20\,\mathrm{N}$, (c) Cart position $x(t)$ revealing uncontrolled drift under single-loop PID, and (d) Pole phase portrait $(\theta, \dot{\theta})$.

7 Project Status and Roadmap

Milestone M2 — Classical + Modern Control — Completed. Library core, four reference dynamical systems, RK4 simulator, PID with anti-windup, StateFeedback (Ackermann), LQR (from-scratch CARE solver), 13 benchmark metrics, multi-controller comparison harness, Experiment 03 (PID unstable stabilization), Experiment 04 (LQR vs. Pole Placement vs. PID on Cart-Pole), CI on Python 3.10–3.12 with 90+ passing unit tests, and complete theory notes for Modules 01–05.

Phase 1 Priorities (Next).

- **State Estimation (`aimct.estimation`):** Luenberger observers and continuous/discrete Kalman filtering.
- **System Identification:** Data-driven linear state-space fitting for L1 and L2 benchmark systems.
- **Nonlinear Control & Swing-Up:** Energy-based swing-up controller for the Cart-Pole to push θ_0 past the linear basin ($\theta_0 = \pi \to 0$).
- **Phase 2 (Optimal Control):** Constrained Model Predictive Control (linear MPC with from-scratch QP solver).

Reproduction Commands.

```
git clone https://github.com/zalihthomas-ui/ai-meets-control-theory.git
cd ai-meets-control-theory
pip install -e ".[dev,xcheck]"
pytest -q
python experiments/03_pid_stabilizes_unstable/run.py
python experiments/04_lqr_vs_pole_placement_cartpole/run.py
```